



Document de référence — Travail en équipe

1. Répartition des tâches

Chaque membre est responsable de son package. Les Pull Requests doivent être validées par au moins un autre membre avant le merge sur main.

Membre	Package principal	Classes assignées	Branche Git
PACHA	model (catalogue)	Catalogue, Morceau, Album, Artiste, Groupe	feature/model-catalogue
ETHAN	model (users)	Utilisateur, Visiteur, Abonne, Administrateur, PlayList	feature/model-users
ELVIRA	controller	MainController, CatalogueController, PlaylistController, UserController	feature/controller
FELIX	view	IView, ConsoleView, GraphicView	feature/view

⚠ Règle importante : on ne touche jamais au package d'un autre sans en parler d'abord. En cas de conflit Git, on règle ça ensemble.

2. Classes, attributs et méthodes

Ce tableau est le contrat de l'équipe. Chaque membre doit respecter les signatures définies ici pour que les packages s'intègrent sans surprise.

Rôle	Nom / Signature	Description
Morceau		
Attributs	String titre	Titre du morceau
	int duree	Durée en secondes
	String genre	Genre musical (Pop, Rock...)
	int annee	Année de sortie
	Artiste artiste	Artiste ou groupe associé
	int nbEcoutes	Compteur d'écoutes global

Rôle	Nom / Signature	Description
Méthodes	void jouer()	Simule la lecture (pause console / barre graphique)
	String toString()	Affichage lisible du morceau
	int getDuree()	Getter durée
Album		
Attributs	String titre	Titre de l'album
	int annee	Année de sortie
	String type	studio / live / compilation
	ArrayList<Morceau> morceaux	Liste des morceaux
Méthodes	void ajouterMorceau(Morceau m)	Ajoute un morceau à l'album
	int getDureeTotal()	Calcule la durée totale
Artiste / Groupe		
Attributs	String nom	Nom de l'artiste ou du groupe
	String biographie	Courte biographie
	ArrayList<Album> albums	Discographie
	ArrayList<Artiste> membres	(Groupe seulement) membres du groupe
Méthodes	ArrayList<Album> getAlbums()	Retourne la discographie
	void ajouterAlbum(Album a)	Ajoute un album
Catalogue		
Attributs	ArrayList<Morceau> morceaux	Tous les morceaux disponibles
	ArrayList<Album> albums	Tous les albums
	ArrayList<Artiste> artistes	Tous les artistes/groupes
Méthodes	ArrayList<Morceau> rechercher(String q)	Recherche par titre/artiste/album
	void ajouterMorceau(Morceau m)	Admin : ajoute un morceau
	void supprimerMorceau(Morceau m)	Admin : supprime un morceau
	int getNbEcoutesTotal()	Stat : nombre total d'écoutes
Utilisateur (abstract)		
Attributs	String login	Identifiant unique
	String motDePasse	Mot de passe (hashé)
	String nom	Nom d'affichage
Méthodes	boolean seConnecter(String l, String mdp)	Vérifie les identifiants
	abstract String getRole()	Retourne le rôle (abstract)

Rôle	Nom / Signature	Description
Abonne extends Utilisateur		
Attributs	ArrayList<PlayList> playlists	Playlists personnelles
	ArrayList<Morceau> historique	Historique d'écoute
Méthodes	void creerPlaylist(String nom)	Crée une nouvelle playlist
	void ajouterAPlaylist(Morceau m, PlayList p)	Ajoute un morceau à une playlist
	void retirerDePlaylist(Morceau m, PlayList p)	Retire un morceau
	ArrayList<Morceau> getHistorique()	Retourne l'historique
PlayList		
Attributs	String nom	Nom de la playlist
	ArrayList<Morceau> morceaux	Morceaux de la playlist
	Abonne propriétaire	Abonné propriétaire
Méthodes	void ajouter(Morceau m)	Ajoute un morceau (vérifie doublon)
	void retirer(Morceau m)	Retire un morceau
	int getDureeTotal()	Durée totale de la playlist
Administrateur extends Utilisateur		
Méthodes	void suspendreCompte(Abonne a)	Suspend un compte abonné
	void supprimerCompte(Abonne a)	Supprime un compte abonné
	void afficherStats(Catalogue c)	Affiche les statistiques globales

3. Relations entre classes (résumé)

Héritage

- Utilisateur (abstract) ← Visiteur
- Utilisateur (abstract) ← Abonne
- Utilisateur (abstract) ← Administrateur

Associations

- Catalogue <—> Morceau (1 catalogue contient N morceaux)
- Catalogue <—> Album (1 catalogue contient N albums)
- Catalogue <—> Artiste (1 catalogue contient N artistes)
- Album <—> Morceau (1 album contient N morceaux)
- Artiste — Morceau (1 artiste chante N morceaux)
- Groupe <—> Artiste (1 groupe contient N artistes membres)
- Abonne <—> PlayList (1 abonné possède N playlists)
- PlayList <—> Morceau (1 playlist contient N morceaux)

Architecture MVC — règle d'or

- Model → ne connaît ni View ni Controller
- Controller → connaît Model ET View
- View → connaît Controller (mais pas Model directement)

4. Conventions de code

Nommage

- Classes : PascalCase → Morceau, PlayList, MainController
- Méthodes & variables : camelCase → getNbEcoutés(), nbEcoutés
- Constantes : UPPER_SNAKE_CASE → MAX_ECOUTES_VISITEUR
- Packages : tout en minuscules → model, view, controller

Commentaires & JavaDoc

Toute classe et toute méthode publique doit avoir une JavaDoc minimale :

```
/** Retourne la durée totale de la playlist en secondes. */  
public int getDureeTotal() { ... }
```

Gestion des exceptions

- Ne jamais laisser passer une NullPointerException silencieusement
- Utiliser des exceptions métier explicites : MorceauDejaPresent, ComptetInexistant...
- Toujours valider les entrées utilisateur dans le Controller, jamais dans le Model

5. Conventions Git

Branches

- main → code stable uniquement, on n'y commit jamais directement
- develop → branche d'intégration (optionnel)
- feature/xxx → branche de travail de chaque membre

Messages de commit

Format : type(scope): description courte en anglais

Type de commit	Format & exemple
Nouvelle fonctionnalité	feat(model): add Morceau class with attributes
Correction de bug	fix(controller): handle null playlist on login
Refactoring	refactor(view): extract menu display to ConsoleView
Documentation	docs: update README with run instructions
Tests	test(model): add unit tests for Catalogue.rechercher()

6. Planning

Semaine	Dates	Objectifs	Livrables
S1	23/03 – 30/03	Diagramme UML, squelettes de classes, setup Git	Repo GitHub initialisé, classes vides
S2	31/03 – 06/04	Implémentation model complet + tests unitaires	Classes model fonctionnelles
S3	07/04 – 13/04	Controllers + Vue console + sérialisation	Version console fonctionnelle
S4	14/04 – 19/04	Vue graphique (Swing/JavaFX) + corrections + rapport	Rendu final + JAR + JavaDoc

 **Deadline finale : 19/04/2026 — Soutenances : semaine du 20/04/2026**

8. Checklist avant le rendu final

- ☐ Toutes les fonctionnalités de base sont opérationnelles (Visiteur, Abonné, Admin)
- ☐ Au moins 2 fonctionnalités supplémentaires implémentées
- ☐ Architecture MVC respectée (pas de System.out dans le Model !)
- ☐ Code indenté et commenté, JavaDoc générée
- ☐ Sérialisation : données sauvegardées/chargées automatiquement
- ☐ Version console fonctionnelle
- ☐ Version graphique (Swing ou JavaFX) fonctionnelle
- ☐ Fichier .jar exécutable
- ☐ README.md complet (instructions d'exécution)
- ☐ Rapport : UML, scénarios, plan de tests, bilan, sources
- ☐ Dossier compressé prêt à déposer